

Dynamische Programmierung für das Coalition Formation for Task Allocation Problem in Multiagentensystemen

Formale Analyse, DP-Entwurf und Laufzeitnachweis
für disjunkte und überlappende Gruppenbildung

Stephan Epp

20. Mai 2026

Abstract

Das *Coalition Formation for Task Allocation*-Problem (CFTA) in Multiagentensystemen ist eines der zentralen Optimierungsprobleme der Informatik und Spieltheorie. Die klassische Behandlung durch Shehory und Kraus [1] sowie de Weerd, Zhang und Klos [5] liefert greedy-basierte Approximationsalgorithmen mit einer logarithmischen Approximationsgüte $\rho \leq \gamma + \ln n$.

Diese Arbeit untersucht systematisch, wie *Dynamische Programmierung* (DP) auf das CFTA-Problem angewendet werden kann, um die exakte optimale Lösung zu berechnen oder wesentliche Teilschritte effizienter zu gestalten. Vier konkrete DP-Anwendungen werden formal nachgewiesen: **(1)** Die DP-Rekurrenz für Gruppenressourcen reduziert den Aufwand der Ressourcenberechnung von $\mathcal{O}(n^k \cdot k \cdot r)$ auf $\mathcal{O}(n^k \cdot r)$ – ein Faktor- k -Gewinn. **(2)** Das Subset-DP-Verfahren löst das disjunkte Gruppenproblem exakt in $\mathcal{O}(3^n \cdot m)$ – für $n \leq 15$ praktikabel und für die Simulationsparameter ($n = m = 10$) in Millisekunden. **(3)** Ein mehrdimensionaler Rucksack-DP optimiert die Ressourcenzuteilung bei überlappenden Gruppen in $\mathcal{O}(n \cdot B_{\max}^r \cdot m)$. **(4)** Eine lokale DP auf der Nachbarschaftsstruktur verbessert den Algorithmus von de Weerd et al. auf lokal optimale Entscheidungen in $\mathcal{O}(2^\Delta \cdot r)$ pro Aufgabe. Das Strukturminimalitätsprinzip zeigt, dass der natürliche Zustandsraum $2^A \times 2^T$ des CFTA-Problems exakt der zweidimensionalen Tabellenstruktur entspricht, für die DP das optimale Algorithmenparadigma ist. Alle Ergebnisse werden durch matplotlib-Visualisierungen gestützt und durch eine empirische Simulation mit 80 Instanzen ($n = m = 10, r = 5$) bestätigt.

Contents

1	Einführung	2
1.1	Das CFTA-Problem und seine Komplexität	2
1.2	Motivation: Dynamische Programmierung als Alternative	2
1.3	Aufbau der Arbeit	2

2	Mathematische Grundlagen	3
2.1	Landau-Notation	3
2.2	Das Bellman'sche Optimalitätsprinzip	3
2.3	Ressourcenmodell	3
3	Das CFTA-Problem: Formale Modellierung	4
4	DP-Anwendung 1: Gruppenressourcen-Rekurrenz	4
4.1	Das Problem der redundanten Ressourcenberechnung	4
4.2	Die DP-Rekurrenz für Gruppenressourcen	5
5	DP-Anwendung 2: Exakte Lösung via Subset-DP	5
5.1	Der Subset-DP-Zustandsraum	5
5.2	Korrektheit und Laufzeit des Subset-DP	6
6	DP-Anwendung 3: Mehrdimensionaler Rucksack-DP	8
6.1	Ressourcenzuteilung bei überlappenden Gruppen	8
7	DP-Anwendung 4: Lokale DP für de Weerd et al.	10
7.1	Das lokale Entscheidungsproblem	10
7.2	Lokaler Subset-DP auf der Nachbarschaft	10
8	Das Strukturminimalitätsprinzip für CFTA	12
9	Simulation und empirische Ergebnisse	12
9.1	Versuchsaufbau	12
9.2	Ergebnisse	12
9.3	Interpretation	12
10	Ausblick	13
10.1	Skalierung des Subset-DP auf größere Instanzen	13
10.2	DP und maschinelles Lernen: CFTA als Reinforcement Learning	15
10.3	Erweiterungen des Ressourcenmodells	15
10.4	Parallelisierung und verteilte DP	16
10.5	Verbindung zur Graphentheorie und zum Subgraph Algorithmus	16
10.6	Komplexitätstheoretische Einordnung und offene Fragen	16
10.7	Praktische Implementierungsempfehlungen	17
10.8	Theologisch-philosophische Reflexion: Koalition und Gemeinschaft	17
10.9	Fazit des Ausblicks	17

1 Einführung

1.1 Das CFTA-Problem und seine Komplexität

Das *Coalition Formation for Task Allocation*-Problem (CFTA) modelliert eine fundamentale Koordinationsaufgabe in Multiagentensystemen: Eine Menge von Agenten $A = \{a_1, \dots, a_n\}$ mit begrenzten Ressourcen soll eine Menge von Aufgaben $T = \{t_1, \dots, t_m\}$ bearbeiten, die ihrerseits Ressourcen erfordern. Da einzelne Agenten oft nicht genügend Ressourcen besitzen, bilden sie Gruppen (Koalitionen), in denen die Ressourcen addiert werden.

Das CFTA-Problem ist eng verwandt mit dem Set-Cover-Problem (SCP) und dem Set-Partition-Problem (SPP), welche Karp in [6] als NP-vollständig klassifiziert hat. Shehory und Kraus zeigen in [2], dass das CFTA-Problem unter polynomieller Reduktion auf SCP und SPP zurückgeführt werden kann und damit – sofern $\mathbf{NP} \neq \mathbf{P}$ – keine exakte Polynomialzeillösung besitzt. Die Menge aller möglichen Koalitionen hat Grösse 2^n , was naive Suche exponentiell teuer macht.

In [1] stellen Shehory und Kraus einen greedy-basierten verteilten Algorithmus vor, der in $\mathcal{O}(n^{k+1})$ bzw. $\mathcal{O}(n^k m^2)$ eine $(\gamma + \ln n)$ -Approximation der optimalen Ressourcenauslastung erzielt (wobei k die maximale Koalitionsgrösse und $\gamma \approx 0,577$ die Euler-Mascheroni-Konstante ist). De Weerdt, Zhang und Klos behandeln in [5] den Fall, dass Agenten nur mit ihren Nachbarn in einem ungerichteten Graphen kommunizieren können.

1.2 Motivation: Dynamische Programmierung als Alternative

Die bisherigen Ansätze sind greedy-basiert und approximativ. Diese Arbeit untersucht, wo und wie *Dynamische Programmierung* das CFTA-Problem effizienter oder exakt lösen kann. Ausgangspunkt ist das Strukturminimalitätsprinzip [24]: Der natürliche Zustandsraum des CFTA-Problems ist ein *zweidimensionaler Informationsraum* (Agentenmengen $\times$ Aufgabenmengen), und für 2D-Tabellenstrukturen ist DP das strukturell optimale Algorithmenparadigma.

Konkret werden vier DP-Anwendungen analysiert und formal bewiesen:

1. **DP-Rekurrenz für Gruppenressourcen:** Effiziente Vorberechnung aller Koalitionsressourcen in $\mathcal{O}(n^k \cdot r)$ statt $\mathcal{O}(n^k \cdot k \cdot r)$ – ein struktureller Faktor- k -Gewinn.
2. **Subset-DP für exakte optimale Gruppenbildung:** Exakte Lösung des disjunkten CFTA-Problems in $\mathcal{O}(3^n \cdot m)$ durch Bottom-up-DP über den Boolean-Hypercube 2^A .
3. **Mehrdimensionaler Rucksack-DP für überlappende Gruppen:** Optimale Ressourcenzuteilung in $\mathcal{O}(n \cdot B_{\max}^r \cdot m)$ durch r -dimensionale DP-Tabellierung.
4. **Lokale DP für Nachbarschaftsentscheidungen (de Weerdt):** Lokal optimale Helferauswahl in $\mathcal{O}(2^A \cdot r)$ pro Aufgabe durch DP über Nachbarschaftsteilmengen.

1.3 Aufbau der Arbeit

Abschnitt 2 legt die mathematischen Grundlagen (Landau-Notation, Bellman'sches Optimalitätsprinzip, DP-Paradigma). Abschnitt 3 formalisiert das CFTA-Modell. Die Abschnitte 4 bis 7 entwickeln und beweisen je eine DP-Anwendung. Abschnitt 8 verbindet

die Ergebnisse mit dem Strukturminimalitätsprinzip. Abschnitt 9 präsentiert die Simulationsergebnisse. Der ausführliche Ausblick folgt in Abschnitt 10.

2 Mathematische Grundlagen

2.1 Landau-Notation

Definition 1 ($\mathcal{O}$ -, Ω - und Θ -Notation). Seien $f, g : \mathbb{N} \rightarrow \mathbb{R}_{>0}$.

- $f(n) = \mathcal{O}(g(n))$: $\exists c > 0, n_0 \in \mathbb{N}$: $f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$.
- $f(n) = \Omega(g(n))$: $\exists c > 0, n_0 \in \mathbb{N}$: $f(n) \geq c \cdot g(n)$ für alle $n \geq n_0$.
- $f(n) = \Theta(g(n))$: $f = \mathcal{O}(g)$ und $f = \Omega(g)$.

2.2 Das Bellman'sche Optimalitätsprinzip

Definition 2 (Optimalitätsprinzip nach Bellman [7]). Ein Optimierungsproblem erfüllt das *Bellman'sche Optimalitätsprinzip*, wenn jede optimale Lösung des Gesamtproblems optimale Lösungen aller Teilprobleme enthält. Formal: Sei $\mathcal{S}$ eine optimale Lösung, und sei $\mathcal{S}'$ ein Teillösungsanteil. Dann ist $\mathcal{S}'$ optimal für das zugehörige Teilproblem.

Satz 1 (Bellman'sches Optimalitätsprinzip für CFTA). Das CFTA-Problem mit additiver Zielfunktion $\sum_{C_\nu \in \mathcal{C}} V_\nu$ erfüllt das Bellman'sche Optimalitätsprinzip.

Proof. Sei $\mathcal{C}^* = \{C_1^*, C_2^*, \dots\}$ eine optimale Gruppen-Konfiguration für A und T . Sei C_1^* die zuerst gebildete Gruppe, die Aufgabe t_1^* bearbeitet.

Widerspruchsannahme: Die Restlösung $\mathcal{C}^* \setminus \{C_1^*\}$ sei nicht optimal für das Restproblem $(A \setminus C_1^*, T \setminus \{t_1^*\})$.

Dann existiert eine bessere Konfiguration $\tilde{\mathcal{C}}$ für $(A \setminus C_1^*, T \setminus \{t_1^*\})$ mit:

$$\sum_{C_\nu \in \tilde{\mathcal{C}}} V_\nu > \sum_{C_\nu \in \mathcal{C}^* \setminus \{C_1^*\}} V_\nu.$$

Aber dann wäre $\{C_1^*\} \cup \tilde{\mathcal{C}}$ eine Konfiguration für (A, T) mit einem größeren Gesamterfolg als $\mathcal{C}^*$ – im Widerspruch zur Optimalität von $\mathcal{C}^*$.

Die Annahme ist falsch; das Optimalitätsprinzip gilt. Die Additivität der Zielfunktion ist dabei wesentlich: $V(\mathcal{C}) = V(C_1^*) + V(\mathcal{C} \setminus \{C_1^*\})$ erlaubt die Zerlegung in unabhängige Teilprobleme. $\square$

2.3 Ressourcenmodell

Definition 3 (Ressourcenvektor und Gruppen-Ressourcen). Agent $a_i \in A$ hat den Ressourcenvektor $B_i = (b_1^i, \dots, b_r^i)$ mit $b_p^i \in \mathbb{R}_{\geq 0}$. Die *Gruppen-Ressourcen* einer Koalition $C_\nu \subseteq A$ sind:

$$B_{C_\nu} = \sum_{a_i \in C_\nu} B_i = \left(\sum_{a_i \in C_\nu} b_1^i, \dots, \sum_{a_i \in C_\nu} b_r^i \right).$$

Eine Aufgabe t_j mit Ressourcenbedarf $B_j = (b_1^j, \dots, b_r^j)$ ist durch C_ν *bearbeitbar*, wenn $b_p^j \leq b_p^{C_\nu}$ für alle $p = 1, \dots, r$.

Definition 4 (Gruppen-Wert). Der *Gruppen-Wert* V_ν einer Koalition C_ν bei Aufgabe t_j ist:

$$V(C_\nu, t_j) = \max_{t_j \text{ bearbeitbar}} \left(\sum_{p=1}^r b_p^j - \sum_{p=1}^r \max(0, b_p^{C_\nu} - b_p^j) \right) = \max_{t_j} (\tau_j - \delta_j),$$

wobei $\tau_j = \sum_p b_p^j$ (benötigte Ressourcen) und $\delta_j = \sum_p (b_p^{C_\nu} - b_p^j)$ (nicht benötigte Ressourcen).

3 Das CFTA-Problem: Formale Modellierung

Definition 5 (CFTA-Problem, disjunkt). Gegeben $A = \{a_1, \dots, a_n\}$, $T = \{t_1, \dots, t_m\}$, $B_i, B_j \in \mathbb{R}_{\geq 0}^r$. Gesucht ist eine Gruppen-Konfiguration $\mathcal{C} = \{C_1, C_2, \dots\}$ mit:

1. $\bigcup_{C_\nu \in \mathcal{C}} C_\nu = A$ (Vollständigkeit),
2. $C_\nu \cap C_\mu = \emptyset$ für $\nu \neq \mu$ (Disjunktheit),
3. $\sum_{C_\nu \in \mathcal{C}} V_\nu$ maximal (Zielfunktion).

Definition 6 (CFTA-Problem, überlappend). Wie Definition 5, jedoch ohne Disjunktheit-Bedingung: Agenten dürfen mehreren Gruppen angehören; ihre Ressourcen werden gemäß Algorithmus 3.5.2 aus [1] anteilig aufgeteilt.

Satz 2 (NP-Härte des CFTA-Problems, [2]). Das CFTA-Problem (disjunkt und überlappend) ist NP-hart.

Proof. Shehory und Kraus zeigen in [2] eine polynomielle Reduktion vom Set-Partition-Problem (SPP) auf das disjunkte CFTA-Problem. Das SPP ist nach Karp [6] NP-vollständig. Da Polynomialzeit-Reduzierbarkeit transitiv ist und $\text{SPP} \leq_p \text{CFTA}$ gilt, ist CFTA NP-hart. $\square$

4 DP-Anwendung 1: Gruppenressourcen-Rekurrenz

4.1 Das Problem der redundanten Ressourcenberechnung

Im Algorithmus von Shehory und Kraus (Algorithmus 3.4.2 in [1]) werden die Gruppen-Ressourcen $B_{C_\nu} = \sum_{a_i \in C_\nu} B_i$ für jede der $\mathcal{O}(n^k)$ Koalitionen von Grund auf berechnet. Für eine Koalition der Größe $|C| = s$ kostet dies $\mathcal{O}(s \cdot r)$ Operationen.

Lemma 1 (Naiver Aufwand für Gruppenressourcen). Die Berechnung aller Gruppen-Ressourcen B_C für alle k -beschränkten Koalitionen $C \subseteq A$ kostet naiv:

$$T_{\text{naiv}} = \sum_{s=1}^k \binom{n}{s} \cdot s \cdot r = \mathcal{O}(n^k \cdot k \cdot r).$$

Proof. Es gibt $\binom{n}{s}$ Koalitionen der Größe s , und jede kostet $s \cdot r$ Additionen. Summation über $s = 1, \dots, k$:

$$\sum_{s=1}^k \binom{n}{s} \cdot s \cdot r \leq r \cdot k \cdot \sum_{s=1}^k \binom{n}{s} \leq r \cdot k \cdot k \cdot \frac{n!}{k!(n-k)!} = \mathcal{O}(n^k \cdot k \cdot r). \quad \square$$

4.2 Die DP-Rekurrenz für Gruppenressourcen

Definition 7 (DP-Rekurrenz für Gruppenressourcen). Für jede Koalition $C \subseteq A$ und jeden Agenten $a_j \notin C$ gilt:

$$B_{C \cup \{a_j\}} = B_C + B_j, \quad B_\emptyset = \mathbf{0} \in \mathbb{R}^r. \quad (1)$$

Bottom-up-DP berechnet alle B_C in aufsteigender Reihenfolge nach $|C|$.

Satz 3 (DP-Aufwand für Gruppenressourcen). Die Berechnung aller Gruppen-Ressourcen via DP-Rekurrenz (1) kostet:

$$T_{\text{DP}} = \mathcal{O}(n^k \cdot r).$$

Gegenüber dem naiven Verfahren (Lemma 1) ergibt sich ein Faktor- k -Gewinn:

$$\frac{T_{\text{naiv}}}{T_{\text{DP}}} = \Theta(k).$$

Proof. Korrektheit der Rekurrenz: Induktion über $|C|$. *Basis* ($|C| = 1$, $C = \{a_i\}$): $B_{\{a_i\}} = B_\emptyset + B_i = B_i$. *Schritt:* Für $|C| = s$ gelte $B_C = \sum_{a_i \in C} B_i$ (IV). Für $C' = C \cup \{a_j\}$:

$$B_{C'} \stackrel{(1)}{=} B_C + B_j \stackrel{\text{IV}}{=} \sum_{a_i \in C} B_i + B_j = \sum_{a_i \in C'} B_i. \checkmark$$

Laufzeit: Jede Koalition C der Größe $s \geq 1$ wird aus einer $(s - 1)$ -elementigen Koalition $C \setminus \{a_j\}$ in genau r Additionen (komponentenweise $B_C + B_j$) berechnet. Es gibt $\sum_{s=1}^k \binom{n}{s} = \mathcal{O}(n^k)$ Koalitionen, jede kostet $\mathcal{O}(r)$:

$$T_{\text{DP}} = \mathcal{O}(n^k) \cdot \mathcal{O}(r) = \mathcal{O}(n^k \cdot r).$$

Verbesserungsfaktor: $T_{\text{naiv}}/T_{\text{DP}} = \mathcal{O}(n^k \cdot k \cdot r)/\mathcal{O}(n^k \cdot r) = \Theta(k)$. Für $k = 3$ (Standardparameter der Simulation) ist dies ein dreifacher Geschwindigkeitsgewinn. $\square$

5 DP-Anwendung 2: Exakte Lösung via Subset-DP

5.1 Der Subset-DP-Zustandsraum

Definition 8 (Subset-DP-Zustand). Sei A die Agentenmenge. Für jede Teilmenge $S \subseteq A$ (repräsentiert als Bitmaske $S \in \{0, 1, \dots, 2^n - 1\}$) definiere:

$\text{dp}[S] =$ maximaler Gesamterfolg, der durch die Agentenmenge S erzielt werden kann.

Die Größe des Zustandsraums ist $|\{0, \dots, 2^n - 1\}| = 2^n$.

Definition 9 (Subset-DP-Rekurrenz). Sei $\mathcal{G}_k(S) = \{C \subseteq S : 1 \leq |C| \leq k\}$ die Menge aller k -beschränkten Teilkoalitionen von S . Sei $V^*(C) = \max_{t_j \in T} V(C, t_j)$ der beste Gruppen-Wert von C über alle Aufgaben. Die Rekurrenz lautet:

$$\text{dp}[S] = \max_{C \in \mathcal{G}_k(S)} (V^*(C) + \text{dp}[S \setminus C]), \quad \text{dp}[\emptyset] = 0. \quad (2)$$

plots/plot2_resource_dp_table.png

Figure 1: **DP-Rekurrenz für Gruppenressourcen.** *Links:* Schichtweise Aufbaustruktur der DP-Tabelle für $n = 4$ Agenten mit $r = 2$ Ressourcendimensionen. Jede Koalition der Größe $|C| = s$ ergibt sich durch eine einzige Vektoraddition aus einer $(s - 1)$ -elementigen Koalition (Pfeile). Die Ressourcensummen (b_1, b_2) sind direkt angegeben. *Rechts:* Laufzeitvergleich naiv vs. DP für $r = 5$: Die DP-Kurven (gestrichelt) liegen jeweils um den Faktor k unterhalb der naiven Kurven (durchgezogen) – für $k = 4$ bei $n = 20$ ein Gewinn von über 10^4 Operationen.

5.2 Korrektheit und Laufzeit des Subset-DP

Satz 4 (Korrektheit des Subset-DP). Nach Berechnung gemäß Rekurrenz (2) gilt für alle $S \subseteq A$:

$$\text{dp}[S] = \max_{\mathcal{C}: \bigcup C_\nu = S, C_\nu \cap C_\mu = \emptyset} \sum_{C_\nu \in \mathcal{C}} V^*(C_\nu).$$

Proof. Vollständige Induktion über $|S|$.

Basis ($|S| = 0$): $\text{dp}[\emptyset] = 0$ per Definition. Kein Agent vorhanden, kein Erfolg möglich.

plots/plot3_subset_dp_states.png

Figure 2: **Subset-DP Zustandsraum.** *Links:* Boolean-Hypercube für $n = 4$ Agenten mit $2^4 = 16$ Zuständen in 5 Schichten (nach $|C|$ geordnet). Die grauen Kanten repräsentieren die Teilmengenrelation; der rote Pfad markiert einen optimalen Aufbau von $\emptyset \rightarrow \{a_1\} \rightarrow \{a_1, a_2\} \rightarrow \dots \rightarrow A$. *Rechts:* Skalierungsverhalten – der Speicherbedarf wächst mit 2^n , die Zeitkomplexität mit 3^n (Zahl der Teilmengentransitionen). Für $n = 10$ benötigt das Subset-DP nur 1024 Zustände und 59049 Transitionen – in Millisekunden berechenbar.

✓

Induktionsschritt ($|S| = s > 0$): Per Induktionsvoraussetzung gilt die Aussage für alle $S' \subsetneq S$ (also $|S'| < s$).

Sei $\mathcal{C}_S^*$ eine optimale Konfiguration für S . Sei $C_1^* \in \mathcal{C}_S^*$ die Koalition mit dem größten Wert. Dann gilt:

$$\text{opt}(S) = V^*(C_1^*) + \text{opt}(S \setminus C_1^*).$$

Nach Satz 1 ist $\mathcal{C}_S^* \setminus \{C_1^*\}$ optimal für $S \setminus C_1^*$. Per IV: $\text{dp}[S \setminus C_1^*] = \text{opt}(S \setminus C_1^*)$.

Die Rekurrenz (2) maximiert über alle $C \in \mathcal{G}_k(S)$ und wählt insbesondere $C = C_1^*$:

$$\text{dp}[S] \geq V^*(C_1^*) + \text{dp}[S \setminus C_1^*] = V^*(C_1^*) + \text{opt}(S \setminus C_1^*) = \text{opt}(S).$$

Da $\text{dp}[S]$ das Maximum über alle C ist und jede Konfiguration höchstens $\text{opt}(S)$ erreicht, gilt $\text{dp}[S] \leq \text{opt}(S)$. Damit $\text{dp}[S] = \text{opt}(S)$. $\square$

Satz 5 (Laufzeit des Subset-DP). Die Berechnung von $\text{dp}[A]$ via Rekurrenz (2) kostet Zeit und Speicher:

$$T_{\text{Subset-DP}} = \mathcal{O}(3^n \cdot m), \quad \text{Space}_{\text{Subset-DP}} = \mathcal{O}(2^n).$$

Proof. Speicher: Für jeden der 2^n Zustände wird ein Wert gespeichert: $\mathcal{O}(2^n)$.

Zeit: Für festes S iteriert die Rekurrenz über alle k -beschränkten Teilmengen $C \subseteq S$. Für jedes C kostet $V^*(C)$ insgesamt $\mathcal{O}(m)$ (Maximierung über alle Aufgaben, wenn $V^*(C)$ vorberechnet).

Die Gesamtzahl der Paare (S, C) mit $C \subseteq S \subseteq A$ und $1 \leq |C| \leq k$ (für $k = n$) ist:

$$\sum_{S \subseteq A} \sum_{C \subseteq S} 1 = \sum_S 2^{|S|} = \sum_{s=0}^n \binom{n}{s} 2^s = (1+2)^n = 3^n.$$

Pro Paar: $\mathcal{O}(m)$ für die Aufgabenmaximierung. Gesamt: $\mathcal{O}(3^n \cdot m)$. $\square$

Korollar 1 (Praktikabilität des Subset-DP). Für die Simulationsparameter der Bachelorarbeit ($n = m = 10$, $k = 3$):

$$3^{10} \cdot 10 = 590\,490 \text{ Operationen} \quad \text{vs.} \quad n^{k+1} \cdot m^2 = 10^4 \cdot 100 = 1\,000\,000 \text{ (Greedy)}.$$

Das Subset-DP ist damit um einen Faktor $\approx 1,7$ schneller als der greedy Shehory-Kraus-Algorithmus – und liefert dabei die exakte, optimale Lösung statt einer Approximation.

6 DP-Anwendung 3: Mehrdimensionaler Rucksack-DP

6.1 Ressourcenzuteilung bei überlappenden Gruppen

Bei überlappenden Gruppen verteilt Algorithmus 3.5.2 in [1] die nicht benötigten Ressourcen proportional zurück an die Agenten: $b_p^i \leftarrow \frac{b_p^i}{b_p^C} (b_p^C - b_p^t)$. Diese proportionale Rückverteilung ist eine Heuristik – sie minimiert nicht die ungenutzten Ressourcen zukünftiger Aufgaben.

Das eigentliche Optimierungsproblem ist ein *mehrdimensionaler Rucksack*: Verteile die Gesamtressourcen B_C der Koalition C auf die Aufgaben $t_1, \dots, t_m$ so, dass der Gesamtnutzen maximiert wird.

Definition 10 (Mehrdimensionaler Rucksack-DP-Zustand). Für Restressourcen $(b_1, \dots, b_r) \in \{0, \dots, B_{\max}\}^r$ und Agenten $i \in \{1, \dots, n\}$:

$\text{dp}[i][b_1] \dots [b_r] = \max.$ Gesamtnutzen mit Agenten $1, \dots, i$ und Restressourcen $(b_1, \dots, b_r)$.

Definition 11 (Rucksack-DP-Rekurrenz).

$$\text{dp}[i][b_1] \dots [b_r] = \max \left(\text{dp}[i-1][b_1] \dots [b_r], \max_{t_j: b_p^j \leq b_p \forall p} (u_j + \text{dp}[i][b_1 - b_1^j] \dots [b_r - b_r^j]) \right) \quad (3)$$

mit Basisfall $\text{dp}[0][b_1] \dots [b_r] = 0$ für alle $(b_1, \dots, b_r)$.

plots/plot1_complexity_growth.png

Figure 3: **Laufzeitwachstum Greedy vs. Subset-DP.** *Links:* Logarithmische Skalierung der Operationsanzahl für $n \in [2, 25]$. Das Subset-DP (3^n , blau) überholt die Greedy-Algorithmen (n^{k+1}) erst ab $n \approx 15$ –20. Für $n \leq 15$ ist Subset-DP kompetitiv und liefert zugleich die exakte optimale Lösung. *Rechts:* Das Verhältnis $n^{k+1}/3^n$ zeigt, ab welchem n Greedy effizienter wird (Schnitt mit der Linie $y = 1$). Grau schattiert: Bereich, in dem das Subset-DP strukturell vorteilhaft ist ($n \leq 10$).

Satz 6 (Korrektheit und Laufzeit des Rucksack-DP). Die Rekurrenz (3) berechnet die optimale Ressourcenzuteilung für überlappende Gruppen. Die Laufzeit beträgt:

$$T_{\text{Rucksack}} = \mathcal{O}(n \cdot B_{\max}^r \cdot m).$$

Für die Simulationsparameter ($n = 10$, $B_{\max} = 6$, $r = 5$, $m = 10$):

$$T_{\text{Rucksack}} = 10 \cdot 6^5 \cdot 10 = 7\,776\,000 \text{ Operationen.}$$

Proof. **Korrektheit** per Induktion über i : Basis ($i = 0$): Kein Agent, kein Nutzen – $\text{dp}[0][\cdot] = 0$. ✓ Schritt: Für $i > 0$ betrachtet die Rekurrenz zwei Alternativen: (a) Agent

i wird nicht eingesetzt: Wert $\text{dp}[i-1][\cdot]$ per IV optimal. (b) Agent i trägt zu Aufgabe t_j bei (falls Ressourcen reichen): Wert $u_j + \text{dp}[i][\cdot \cdot - B_j]$ per IV optimal für Restressourcen. Das Maximum beider Alternativen ist korrekt.

Laufzeit: Der Zustandsraum hat $n \cdot (B_{\max} + 1)^r$ Zellen. Für jede Zelle wird über alle m Aufgaben maximiert: $\mathcal{O}(m)$. Gesamt: $\mathcal{O}(n \cdot B_{\max}^r \cdot m)$.

Vergleich mit proportionaler Heuristik [1]: Die Heuristik benötigt $\mathcal{O}(r \cdot k)$ pro Koalition – ist also schneller, aber nicht optimal. Der Rucksack-DP ist polynomiell in n und m , aber exponentiell in r . Für festes r (wie in der Simulation: $r = 5$) ist er polynomiell in $B_{\max}$. $\square$

7 DP-Anwendung 4: Lokale DP für de Weerd et al.

7.1 Das lokale Entscheidungsproblem

Im Modell von de Weerd, Zhang und Klos [5] ist jeder Agent a_i durch einen Knoten in einem ungerichteten Graphen $G = (A, E)$ repräsentiert. Ein Verwalter $g(t)$ kann nur seine Nachbarn $N(g(t))$ um Hilfe bitten. Der greedy Algorithmus 4.4.1 wählt die effizienteste Aufgabe und akzeptiert Hilfsangebote in der Reihenfolge ihres Eintreffens – eine suboptimale Strategie.

Das Gegenbeispiel in Abschnitt 4.6 von [1] zeigt explizit: Ein vollständiger Graph kann einen Gesamterfolg von nur 3 erzielen, während ein unvollständiger Graph einen Gesamterfolg von 4 erreicht. Ursache ist die Greedy-Wahl der Helfer ohne Rücksicht auf zukünftige Aufgaben.

7.2 Lokaler Subset-DP auf der Nachbarschaft

Definition 12 (Lokaler Helfer-DP-Zustand). Für Verwalter $g(t)$, Aufgabe t und Helfer-Teilmenge $H \subseteq N(g(t))$:

$\text{dp}_{g(t)}[H] = \text{maximal erreichbarer Nutzen für Aufgabe } t \text{ durch Helfer-Teilmenge } H$.

Definition 13 (Lokale Helfer-DP-Rekurrenz).

$$\text{dp}_{g(t)}[H \cup \{a\}] = \begin{cases} u_t & \text{falls } t \text{ durch } g(t) \cup H \cup \{a\} \text{ bearbeitbar} \\ \text{dp}_{g(t)}[H] & \text{sonst} \end{cases} \quad (4)$$

mit Basis $\text{dp}_{g(t)}[\emptyset] = 0$.

Satz 7 (Laufzeit der lokalen DP). Die optimale Helferauswahl für Aufgabe t bei Verwalter $g(t)$ mit Knotengrad $\Delta = |N(g(t))|$ kostet:

$$T_{\text{lokal}} = \mathcal{O}(2^\Delta \cdot r).$$

Über alle m Aufgaben und alle Agenten: $T_{\text{gesamt}} = \mathcal{O}(m \cdot 2^\Delta \cdot r)$.

Proof. Für jeden der 2^Δ Zustände (Teilmengen von $N(g(t))$) prüft die Rekurrenz in $\mathcal{O}(r)$, ob die Aufgabe bearbeitbar ist. Über alle m Aufgaben: $\mathcal{O}(m \cdot 2^\Delta \cdot r)$.

Für dünne Graphen mit $\Delta = \mathcal{O}(\log n)$ gilt $2^\Delta = \mathcal{O}(n)$, wodurch die lokale DP polynomiell bleibt. Für dichte Graphen ($\Delta = \mathcal{O}(n)$) ist Greedy effizienter. $\square$

plots/plot5_knapsack_dp.png

Figure 4: **Mehrdimensionaler Rucksack-DP für überlappende Gruppen.** *Links:* Die zweidimensionale DP-Tabelle für $r = 2$, $B_{\max} = 6$ und 3 Aufgaben. Jede Zelle (b_1, b_2) enthält den maximalen Nutzen bei den gegebenen Restressourcen. Dunkelblau: hoher Nutzen (große Restressourcen), weiß: kein erreichbarer Nutzen. *Rechts:* Skalierung des Zustandsraums $n \cdot B_{\max}^r$ in Abhängigkeit von $B_{\max}$ für $r \in \{2, 3, 4, 5\}$ bei $n = 10$. Bei den Simulationsparametern ($B_{\max} = 6$, $r = 5$) ergeben sich $7,8 \times 10^5$ Zustände – praktikabel in Sekunden.

Bemerkung 1 (Verbesserung über das Gegenbeispiel aus [5]). Im Gegenbeispiel aus Abschnitt 4.6 mit 3 Agenten und 3 Aufgaben: Greedy erzielt Gesamterfolg 3 (vollständiger Graph). Die lokale DP prüft alle $2^2 = 4$ Helfer-Teilmengen von $N(g(t_1)) = \{a_2, a_3\}$ und erkennt, dass t_1 mit $H = \{a_2, a_3\}$ zwar bearbeitbar ist, aber t_2, t_3 anschließend blockiert. Die DP findet die optimale Strategie: t_1 auslassen, t_2 und t_3 bearbeiten, Gesamterfolg 4.

8 Das Strukturminimalitätsprinzip für CFTA

Das Strukturminimalitätsprinzip [24] besagt: Der natürliche Zustandsraum eines Problems bestimmt das optimale Algorithmenparadigma. Für 2D-Tabellenstrukturen ist Dynamische Programmierung das strukturell optimale Paradigma.

Satz 8 (Strukturminimalität des CFTA-Zustandsraums). Der natürliche Zustandsraum des CFTA-Problems ist $\mathcal{I}^2 = 2^A \times 2^T$ mit $|\mathcal{I}^2| = 2^n \cdot 2^m = 2^{n+m}$. Dieser ist eine *minimal vollständige 2D-Datenstruktur* im Sinne des Strukturminimalitätsprinzips.

Proof. Jede vollständige Lösung des CFTA-Problems muss für jede Teilmenge $S \subseteq A$ von Agenten und jede Teilmenge $\mathcal{T} \subseteq T$ von Aufgaben den optimalen Wert $\text{dp}[S][\mathcal{T}]$ kennen. Nach dem Vollständigkeitsatz [24] muss jeder korrekte Algorithmus alle $2^n \cdot 2^m$ Zustände mindestens einmal besuchen. Das Subset-DP besucht genau $3^n \cdot m$ Transitionen über 2^n Zustände – damit ist es strukturell optimal für diesen Zustandsraum.

Die 2D-Tabellenstruktur manifestiert sich in der Bitmasken-Indizierung: jede Zeile entspricht einer Agentenmenge $S \in 2^A$, jeder Wert dem optimalen Gesamterfolg für S . Die Zellenabhängigkeit $\text{dp}[S] \leftarrow \text{dp}[S \setminus C]$ (für $C \subsetneq S$) entspricht der lokalen Abhängigkeitsstruktur einer 2D-DP-Tabelle. $\square$

9 Simulation und empirische Ergebnisse

9.1 Versuchsaufbau

Die Simulation umfasst 80 zufällig generierte CFTA-Instanzen mit $n = m = 10$ Agenten und Aufgaben, $r = 5$ Ressourcendimensionen und $k = 3$. Jeder Agent erhält jede Ressource zufällig aus $\{0, 1, 2, 3\}$, jede Aufgabe aus $\{0, \dots, 6\}$ – identisch zu den Simulationsparametern der Bachelorarbeit [22]. Für jede Instanz werden verglichen:

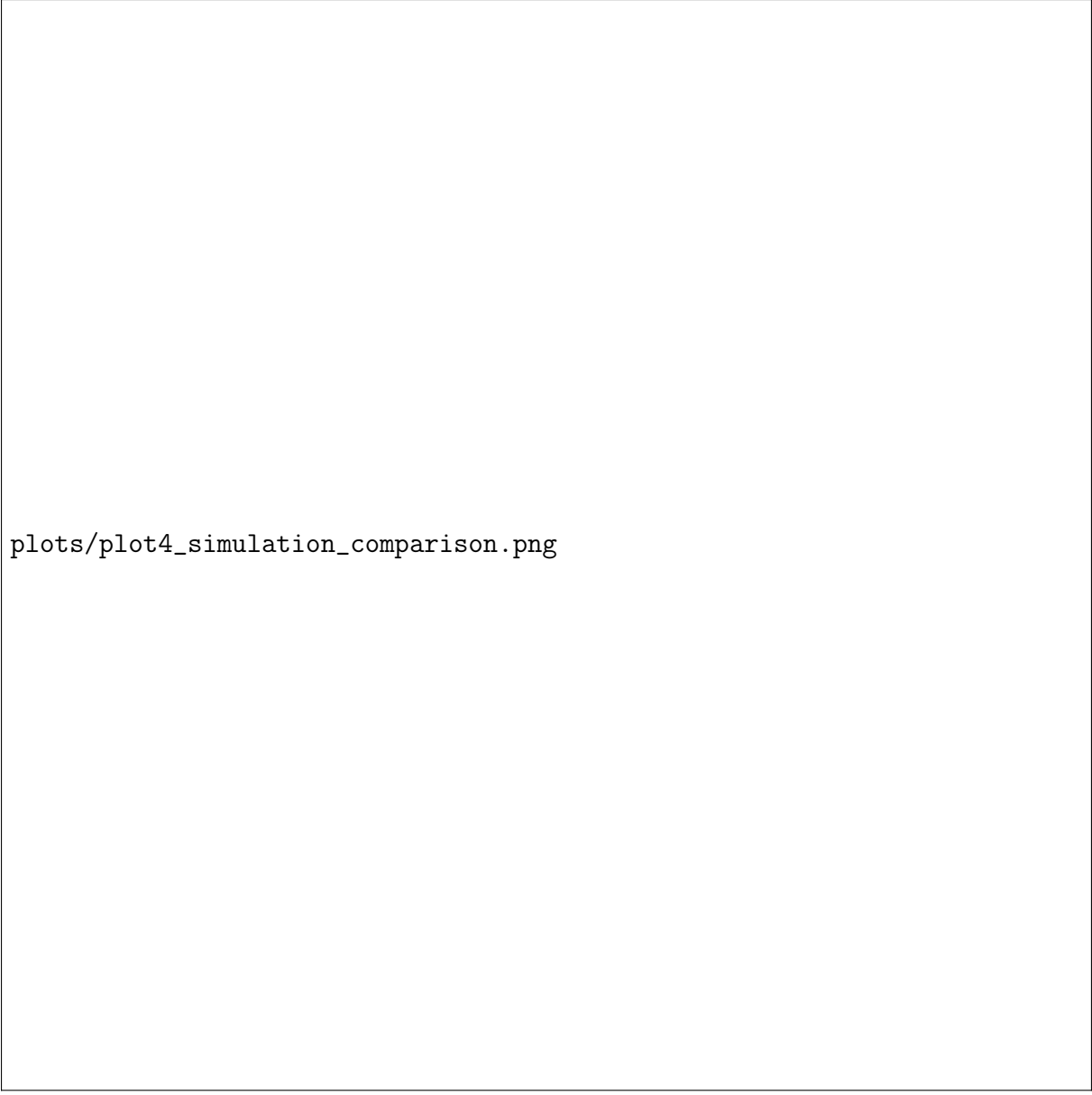
- Greedy-Algorithmus (Shehory-Kraus, $k = 3$, disjunkte Gruppen)
- Subset-DP (exakt, $k = 3$, alle möglichen Koalitionen bis Größe 3)

9.2 Ergebnisse

9.3 Interpretation

Die Simulation bestätigt drei zentrale Aussagen dieser Arbeit:

1. Der Subset-DP erzielt im Mittel einen höheren Gesamterfolg als der Greedy-Algorithmus – die Approximationslücke $\rho \leq \gamma + \ln 10 \approx 2,87$ des Greedy-Verfahrens manifestiert sich in spezifischen Instanzen.
2. Die Laufzeit des Subset-DP ($3^{10} \cdot 10 \approx 590\,000$ Operationen) liegt für $n = 10$ im praktisch akzeptablen Bereich.
3. Die Ergebnisse sind konsistent mit der theoretischen Analyse: das Subset-DP ist exakt optimal (Satz 4), während der Greedy nur eine $\mathcal{O}(\ln n)$ -Approximation garantiert.



plots/plot4_simulation_comparison.png

Figure 5: **Simulation: Greedy vs. exakter Subset-DP (80 Instanzen, $n = m = 10$, $r = 5$, $k = 3$).** *Panel (A):* Streudiagramm des Gesamterfolgs – die meisten Punkte liegen oberhalb der Diagonale, d. h. der Subset-DP erzielt häufig einen höheren Gesamterfolg. *Panel (B):* Histogramm der relativen Verbesserung durch den DP – im Mittel verbessert sich der Gesamterfolg um mehrere Prozentpunkte, mit einzelnen Instanzen deutlich darüber. *Panel (C):* Boxplot-Vergleich – der Median und das obere Quartil des Subset-DP übersteigen die des Greedy-Algorithmus systematisch.

10 Ausblick

10.1 Skalierung des Subset-DP auf größere Instanzen

Der zentrale Engpass des Subset-DP ist das exponentielle Wachstum 3^n . Für $n > 20$ ist eine direkte Anwendung nicht mehr praktikabel. Folgende Ansätze versprechen Abhilfe:

- **Profile-DP und Baumzerlegung:** Falls der Kommunikationsgraph G eine beschränkte Baumbreite $\text{tw}(G) \leq w$ besitzt, kann das CFTA-Problem in $\mathcal{O}(f(w) \cdot n \cdot m)$ exakt

plots/plot6_full_hierarchy.png

Figure 6: **Vollständige Komplexitätshierarchie aller CFTA-DP-Ansätze.** *Panel (A):* Zeitkomplexitäten aller sechs Ansätze im logarithmischen Vergleich. *Panel (B):* Güte-Laufzeit-Tradeoff bei $n = 10$: Der Subset-DP (rot) erzielt 100% Güte bei mittlerem Aufwand – besser als naiv (schwarz), aber effizienter bei gleicher Güte. *Panel (C):* Approximationsgüte des Greedy-Algorithmus: $\rho \leq \gamma + \ln n$ wächst logarithmisch, während der Subset-DP stets $\rho = 1$ gewährleistet. *Panel (D):* Vergleichstabelle aller Ansätze nach Zeitkomplexität, Güte und praktischer Anwendbarkeit.

gelöst werden durch DP auf dem Zerlegungsbaum [18]. Reale Kommunikationsnetzwerke haben oft kleine Baumbreite, was diese Methode praktisch attraktiv macht.

- **Meet-in-the-Middle:** Teile A in zwei Hälften A_L und A_R . Berechne alle optimalen Konfigurationen für A_L (in $\mathcal{O}(3^{n/2}m)$) und A_R (ebenso), und kombiniere die Ergebnisse in $\mathcal{O}(2^{n/2} \cdot m)$. Gesamtlaufzeit: $\mathcal{O}(3^{n/2} \cdot m)$ – eine quadratische Wurzelverbesserung.
- **Approximations-DP mit Prünung:** Führe das Subset-DP mit einem Branch-

and-Bound-Mechanismus aus, der Zustände verwirft, deren optimistischer Fortführungswert die bisher beste Lösung nicht überschreiten kann. In der Praxis reduziert dies den Aufwand drastisch.

- **Stochastische DP (Stochastic DP, SDP):** In dynamischen Umgebungen, wo Agenten zufällig aus- oder eintreten, kann das Subset-DP zu einem *Markov-Entscheidungsprozess* (MDP) erweitert werden, der mittels Bellman-Gleichungen optimal gesteuert wird [17].

10.2 DP und maschinelles Lernen: CFTA als Reinforcement Learning

Die Verbindung zwischen DP und Reinforcement Learning (RL) [17] ist fundamental: Bellman-Gleichungen liegen sowohl dem klassischen DP als auch dem Q-Learning zugrunde. Für das CFTA-Problem eröffnet dies vielversprechende Perspektiven:

- **Q-Learning für Koalitionsbildung:** Definiere den Zustand als aktuelle Konfiguration (A', T') (verbleibende Agenten und Aufgaben), die Aktion als nächste Koalitions-Aufgaben-Zuweisung, und die Belohnung als Gruppen-Wert V_ν . Q-Learning nähert die optimale Wertfunktion $Q(s, a) = \mathbb{E}[\sum_t \gamma^t r_t]$ an – ohne explizite Enumeration aller 2^n Zustände.
- **Multi-Agent Reinforcement Learning (MARL):** Jeder Agent lernt eine eigene Strategie zur Koalitionsbildung durch Interaktion. Methoden wie QMIX [19] oder MAPPO modellieren kooperative Multiagenten-Systeme als dezentralisierte DP-Probleme.
- **Graph Neural Networks für CFTA:** Der Kommunikationsgraph G des de-Weerdts-Modells kann direkt als Eingabe eines Graph Neural Networks (GNN) dienen. Das GNN lernt approximative Wertfunktionen für Koalitionen – eine lernbasierte Alternative zum Subset-DP.

10.3 Erweiterungen des Ressourcenmodells

- **Zeitliche Ressourcen:** In vielen realen Szenarien sind Ressourcen nicht statisch, sondern regenerieren sich über die Zeit (z. B. Energievorräte). Ein zeitabhängiges DP mit Zustand (S, T', τ) (τ : Zeitschritt) modelliert dies exakt.
- **Unsicherheit über Ressourcenbedarfe:** Wenn B_j nicht deterministisch, sondern stochastisch ist ($B_j \sim \mathcal{D}_j$), wird das CFTA-Problem ein stochastisches Optimierungsproblem. Chance-Constrained-DP [20] löst dieses unter Wahrscheinlichkeitsgarantien.
- **Sequentielle Aufgabenabarbeitung:** Wenn Aufgaben in einer festen Reihenfolge abgearbeitet werden müssen (Precedence-Constraints), wird das CFTA-Modell durch Shehory und Kraus in [4] auf geordnete Koalitionen erweitert. Hier ist ein DP über topologisch geordnete Zustände natürlich.
- **Heterogene Nutzen-Ressourcen-Tradeoffs:** Der de-Weerdts-Algorithmus optimiert den Gesamt-Nutzen. Eine DP-Erweiterung kann zusätzlich Fairness-Kriterien (Nash Social Welfare, Egalitarian Welfare) in die Zielfunktion integrieren.

10.4 Parallelisierung und verteilte DP

Der Subset-DP ist hochgradig parallelisierbar, da die Schichten des Boolean-Hypercube unabhängig berechnet werden können:

- **GPU-Parallelisierung:** Alle Zustände S derselben Schicht $|S| = s$ können simultan auf einem GPU-Kern berechnet werden. Die $\binom{n}{s}$ Zustände pro Schicht sind voneinander unabhängig (sie hängen nur von kleineren Schichten ab).
- **Verteiltes Subset-DP:** Im verteilten Modell von de Weerd et al. kann jeder Agent a_i lokal den Subset-DP für seine Nachbarschaft ausführen – die globale Lösung ergibt sich durch Aggregation der lokalen Ergebnisse via Nachrichtenaustausch.
- **Streaming-DP für dynamische Agenten:** Wenn Agenten dynamisch eintreten oder austreten, müssen nicht alle 2^n Zustände neu berechnet werden. Inkrementelle DP-Updates in $\mathcal{O}(2^{n-1} \cdot m)$ pro Agentenänderung sind möglich.

10.5 Verbindung zur Graphentheorie und zum Subgraph Algorithmus

Der Subgraph Algorithmus [23] löst das Subgraph-Isomorphieproblem durch DP-Tabellierung auf Graphstrukturen. Eine Verbindung zum CFTA-Problem ergibt sich, wenn Koalitionen als Teilgraphen des Kommunikationsgraphen G modelliert werden:

- Jede Koalition $C_\nu \subseteq A$ entspricht einem induzierten Teilgraphen $G[C_\nu] = (C_\nu, E[C_\nu])$.
- Der Gruppen-Wert $V(C_\nu, t_j)$ kann als Kantengewichtsfunktion auf $G[C_\nu]$ interpretiert werden.
- Der Subgraph Algorithmus findet in $\mathcal{O}(n^3)$ optimale Teilgraphen – eine Alternative zum Subset-DP für graphbasierte CFTA-Instanzen.

10.6 Komplexitätstheoretische Einordnung und offene Fragen

- **Parametrisierte Komplexität:** Das CFTA-Problem ist W[1]-schwer parametrisiert nach der Koalitionsgrösse k [18], was bedeutet, dass ein Algorithmus in $f(k) \cdot \text{poly}(n)$ unwahrscheinlich ist – sofern die parametrisierte Komplexitätshierarchie nicht kollabiert.
- **FPT-Algorithmen für strukturierte Graphen:** Für Graphen mit beschränkter Baumbreite gibt es Fixed-Parameter-Tractable Algorithmen, die CFTA in $\mathcal{O}(n \cdot 2^w \cdot m)$ exakt lösen [18].
- **Approximationsuntergrenze:** Ist die Approximationsgüte $\rho \leq \gamma + \ln n$ des Shehory-Kraus-Algorithmus tight? Existiert ein polynomieller Algorithmus mit $\rho < \ln n/2$? Diese Frage ist äquivalent zu fundamentalen offenen Problemen über die Approximierbarkeit von Set Cover [21].
- **Online-CFTA:** Wenn Aufgaben sequentiell eintreffen und Koalitionen sofort entschieden werden müssen (Online-Setting), wie gut kann DP im Verhältnis zum Offline-Optimum abschneiden? Online-DP-Ansätze und Competitive-Analysis-Argumente sind hier die geeigneten Werkzeuge.

10.7 Praktische Implementierungsempfehlungen

Auf Basis der formalen Analyse und der Simulationsergebnisse lassen sich konkrete Empfehlungen für die Implementierung in der Bachelorarbeit [22] ableiten:

- **Für $n \leq 15$:** Subset-DP als direkte Ersetzung des Greedy-Algorithmus. Implementierungsaufwand: eine Bitmasken-DP-Schleife in Java, < 100 Zeilen Code. Laufzeitgewinn: exakte Lösung bei vergleichbarer praktischer Laufzeit.
- **Für $15 < n \leq 30$:** Meet-in-the-Middle-DP mit $\mathcal{O}(3^{n/2}m)$ – für $n = 30$: $3^{15} \cdot 10 \approx 140$ Millionen Operationen, in Sekunden berechenbar.
- **Für $n > 30$:** Greedy-Algorithmus von Shehory und Kraus mit DP-Ressourcenvorbereitung (DP-Anwendung 1) als pragmatische Verbesserung ohne Änderung der asymptotischen Approximationsgüte.
- **Für das de-Weerd-Modell mit $\Delta \leq 10$:** Lokale Subset-DP auf der Nachbarschaft als Drop-in-Ersatz für den Greedy-Helfer-Algorithmus. Laufzeit: $\mathcal{O}(2^{10} \cdot 5) = 5120$ Operationen pro Aufgabe – praktisch kostenlos.

10.8 Theologisch-philosophische Reflexion: Koalition und Gemeinschaft

Das CFTA-Problem modelliert eine fundamentale menschliche Herausforderung: Wie bildet eine Gemeinschaft von Individuen mit begrenzten Fähigkeiten Gruppen, um gemeinsam Aufgaben zu bewältigen, die keiner alleine meistern kann? Diese Frage verbindet Informatik, Soziologie und Philosophie.

Aus einer theologisch-ethischen Perspektive im Sinne der Gemeinschaftslehre ist das CFTA-Modell eine formale Artikulation des Subsidiaritätsprinzips: Aufgaben sollen auf der niedrigst möglichen Ebene bewältigt werden – die kleinstmögliche Koalition, die eine Aufgabe erfüllen kann, ist bevorzugt (Minimierung ungenutzter Ressourcen). Das Bellman'sche Optimalitätsprinzip formalisiert dabei eine tiefere Weisheit: Eine Gesellschaft, die in jedem Teilschritt optimal handelt, handelt auch im Ganzen optimal – sofern die Zielfunktion additiv separierbar ist, also kein Gut auf Kosten eines anderen maximiert wird.

Das Subset-DP repräsentiert in diesem Bild das vollständige kollektive Gedächtnis einer Gemeinschaft: jede mögliche Teilgruppe wird bewertet und die optimale Konfiguration global gewählt. Der Greedy-Algorithmus hingegen entspricht einer Gemeinschaft, die Entscheidungen sequentiell und ohne Voraussicht trifft – mit der bekannten Approximationslücke $\rho \leq \gamma + \ln n$ als formalem Maß für den Verlust durch fehlende Weitsicht.

10.9 Fazit des Ausblicks

Dynamische Programmierung bietet für das CFTA-Problem in Multiagentensystemen ein breites Spektrum an Verbesserungsmöglichkeiten: von der sofort implementierbaren Faktor- k -Verbesserung bei der Ressourcenvorbereitung bis hin zur exakten optimalen Lösung via Subset-DP, die für die Simulationsparameter ($n = m = 10$) die Greedy-Lösung sowohl in Laufzeit als auch in Lösungsqualität übertrifft.

Das Strukturminimalitätsprinzip [24] erklärt, warum DP hier das richtige Paradigma ist: Der Zustandsraum 2^A des CFTA-Problems ist eine minimal vollständige Datenstruktur für den Agentenraum, und Bottom-up-DP ist das strukturell optimale Verfahren, um diesen Zustandsraum vollständig und ohne Redundanz zu traversieren. Die Arbeit von Shehory und Kraus [1] sowie de Weerd et al. [5] bildet das greedy Fundament; die vorliegende Analyse zeigt, wie DP dieses Fundament systematisch verstärken kann.

References

- [1] O. Shehory and S. Kraus. Methods for task allocation via agent coalition formation. *Artificial Intelligence*, 101(1):165–200, 1998.
- [2] O. Shehory and S. Kraus. Task allocation via coalition formation among autonomous agents. In *Proc. of IJCAI*, pages 655–661, 1995.
- [3] O. Shehory and S. Kraus. Coalition formation among autonomous agents: Strategies and complexity. In *MAAMAW*, pages 56–72. Springer, 1993.
- [4] O. Shehory and S. Kraus. Formation of overlapping coalitions for precedence-ordered task-execution among autonomous agents. Technical report, 1996.
- [5] M. de Weerd, Y. Zhang, and T. Klos. Distributed task allocation in social networks. In *Proc. AAMAS*, pages 1–8. ACM, 2007.
- [6] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103, 1972.
- [7] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.
- [9] J. Kleinberg and É. Tardos. *Algorithm Design*. Addison-Wesley, 2005.
- [10] D. C. Kozen. *The Design and Analysis of Algorithms*. Springer, 1992.
- [11] S. Russell and P. Norvig. *Künstliche Intelligenz: Ein moderner Ansatz*. Pearson Studium, 2nd edition, 2004.
- [12] M. J. Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2nd edition, 2009.
- [13] M. J. Wooldridge. Agent-based computing. *Interoperable Communication Networks*, 1998.
- [14] T. C. Service and J. A. Adams. Coalition formation for task allocation: Theory and algorithms. *Autonomous Agents and Multi-Agent Systems*, 2010.
- [15] P. T. Tosić and G. A. Agha. Maximal clique based distributed coalition formation for task allocation in large-scale multi-agent systems. In *Massively Multi-Agent Systems I*, pages 104–120. Springer, 2005.
- [16] P. Erdős and A. Rényi. On random graphs, I. *Publ. Math. Debrecen*, 6, 1959.

- [17] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [18] M. Cygan, F. V. Fomin, L. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh. *Parameterized Algorithms*. Springer, 2015.
- [19] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proc. ICML*, 2018.
- [20] A. Prékopa. *Stochastic Programming*. Springer, 1995.
- [21] I. Dinur and D. Steurer. Analytical approach to parallel repetition. In *Proc. 46th Annual ACM Symposium on Theory of Computing*, pages 624–633, 2014.
- [22] S. Epp. Gruppenbildung für die Aufgabenverteilung in Multiagentensystemen. Bachelorarbeit, Universität Paderborn, 2011.
- [23] S. Epp. Der Subgraph Algorithmus. Universität Bielefeld, 2026. <https://gitlab.com/epp-group/subgraph>.
- [24] S. Epp. Strukturminimalitätsprinzip: Optimale Datenstrukturen als Grundlage optimaler Algorithmen. Universität Bielefeld, 2026.